

Interactive Reading Material – PRD

1. What we're building

Problem statement

Learners read static Markdown reading material with no interaction, no visual reinforcement, no way to check understanding, and no way to ask doubts. Engagement and retention are low, and there is no signal on which concepts learners struggle with. Manually upgrading each doc to be interactive is slow, inconsistent, and does not scale across the content backlog.

Pain removed (Session 1 "shift the burden"): the repeated burden of *manually* writing analogies, quiz questions, and explainers for every concept in every doc — and the synthesis/coordination of keeping them updated when source content changes.

Creative differentiators (what makes it stand out)

These are the additions beyond your original idea that drive real user/business value:

- **Misconception-aware remediation.** Don't just mark answers wrong — design distractors so each wrong option maps to a *known misconception*, then route the learner back to the right layer. This is the pedagogical magic and the hardest thing for competitors to copy.
- **Eval-governed generation.** Every generated analogy/MCQ is scored by a rubric before it ships (faithfulness, no hallucination, distractor quality, difficulty). This is the "AI Evals" rigor from Session 2 applied to content, not just to an agent.
- **Instructor analytics (V2).** A concept-difficulty heatmap across a cohort — directly serves the Session 1 "Academics & Analytics" team. Institutions *pay* for analytics; this is your monetization wedge.
- **Source-linked auto-refresh (V2).** When the source MD changes, regenerate only the affected concept units (eval-gated), so content never goes stale.

Confidence-calibrated quizzes (V2). Ask "how sure are you?" to surface over/under-confidence — strong learning-science signal.

2. The LangChain decision —

V1: do NOT use LangChain as your framework. Use plain Python + the LLM SDK (Anthropic / OpenAI / Gemini).

Why not LangChain for V1:

- V1 is a mostly-**linear** pipeline. LangChain's chain/abstraction layer adds indirection and hidden behavior you don't need yet.
- Plain Python is **easier to debug, easier to write evals against, and gives you full control** — which matters because evals are the core of this project.
- Fewer moving parts = more likely to actually ship in a week.

When you DO add a framework (V1.5+): the moment you have **loops, state, human-in-the-loop interrupts, and checkpoints** (the self-evolving loop + the runtime tutor), use **LangGraph** — **not LangChain**. LangGraph models exactly that: graph + state + conditional edges + interrupts.

LangChain's only allowed role: you *may* borrow one or two small **utilities** (e.g. a text splitter, a retriever wrapper) inside your Python without adopting the framework. Optional, not required.

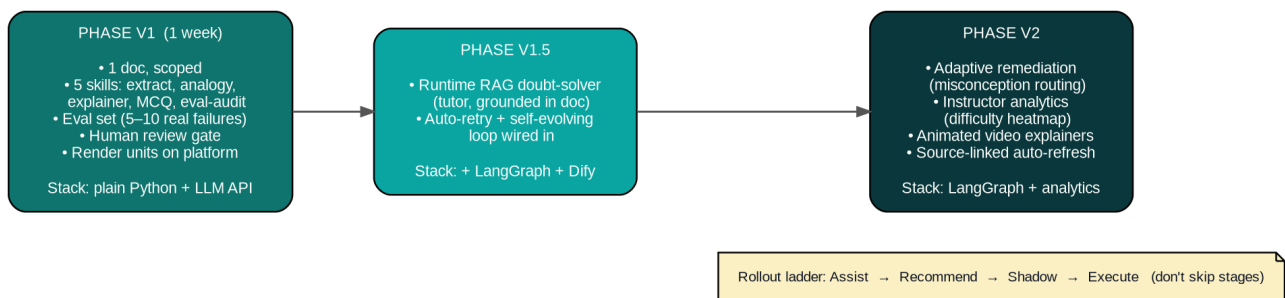
Per-step verdict:

Step	Use this in V1	LangChain?
Parse & chunk doc	Plain Python (or LangChain <code>RecursiveCharacterTextSplitter</code> if handy)	optional utility only
Concept extraction	Direct LLM API call	no
Retrieval (RAG)	Simple vector store (FAISS/Chroma) or keyword span lookup	optional utility only
Analogy / explainer / MCQs	Direct LLM API calls	no
Assemble + schema validate	Python + pydantic	no
Code graders	Plain Python	no
LLM-as-judge	Direct LLM API call	no

Orchestrator	Plain Python script	no (LangGraph later)
Human gate	Spreadsheet / simple UI	no
Logging	Spreadsheet → DB	no

Lock this in: V1 = plain Python, no LangChain framework. V1.5+ = LangGraph. LangChain = optional helper utilities only, never the backbone.

3. Phases (the roadmap)



- **V1 (1 week)** — 1 doc; 5 skills; eval set; human gate; render. Stack: plain Python + LLM API.
- **V1.5** — runtime RAG doubt-solver (tutor); auto-retry + self-evolving loop. Stack: + LangGraph + Dify.
- **V2** — adaptive remediation; instructor analytics; animated video; auto-refresh.

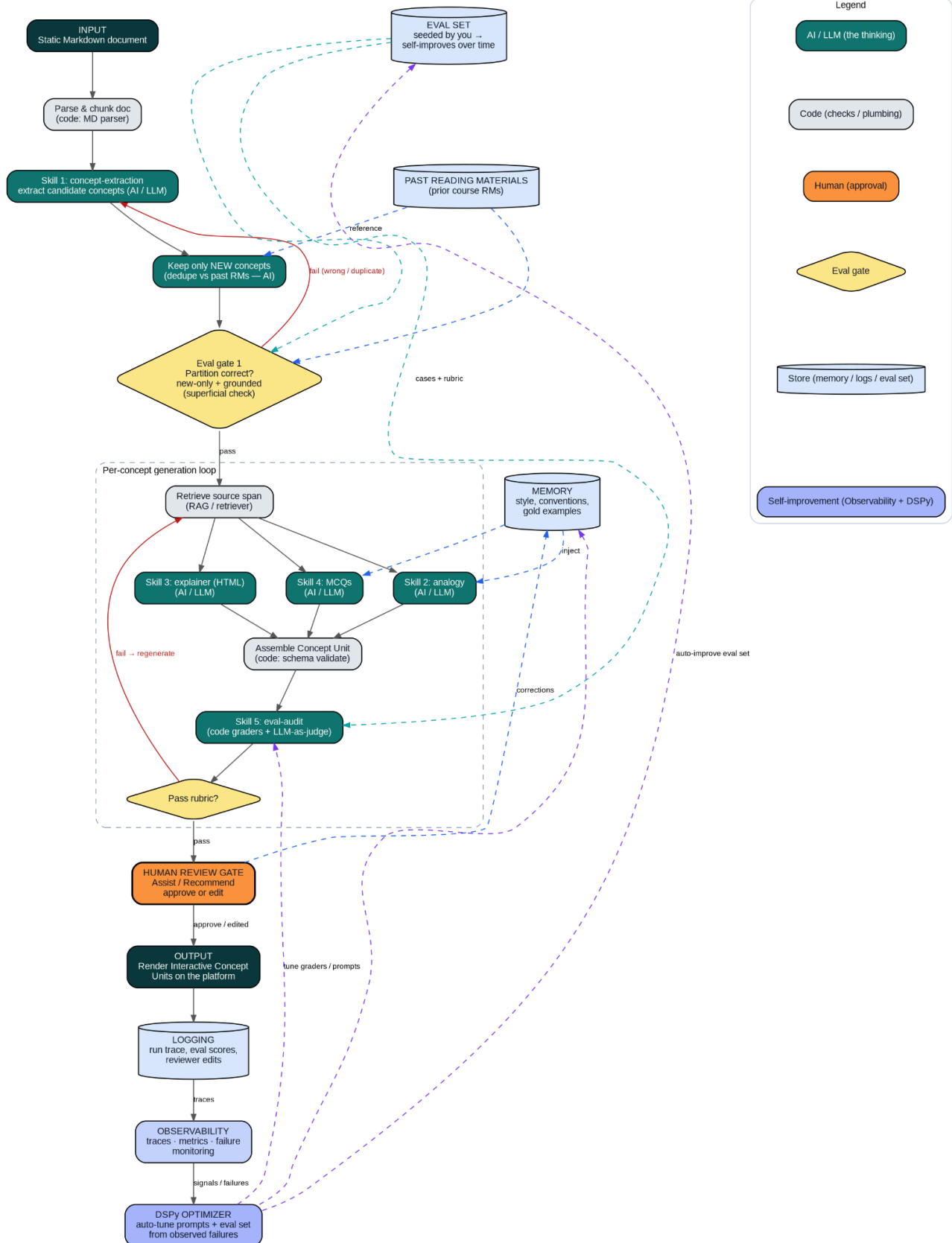
Rollout ladder for autonomy: **Assist** → **Recommend** → **Shadow** → **Execute** (don't skip stages; V1 stays at Assist/Recommend).

4. The workflow (input → output)

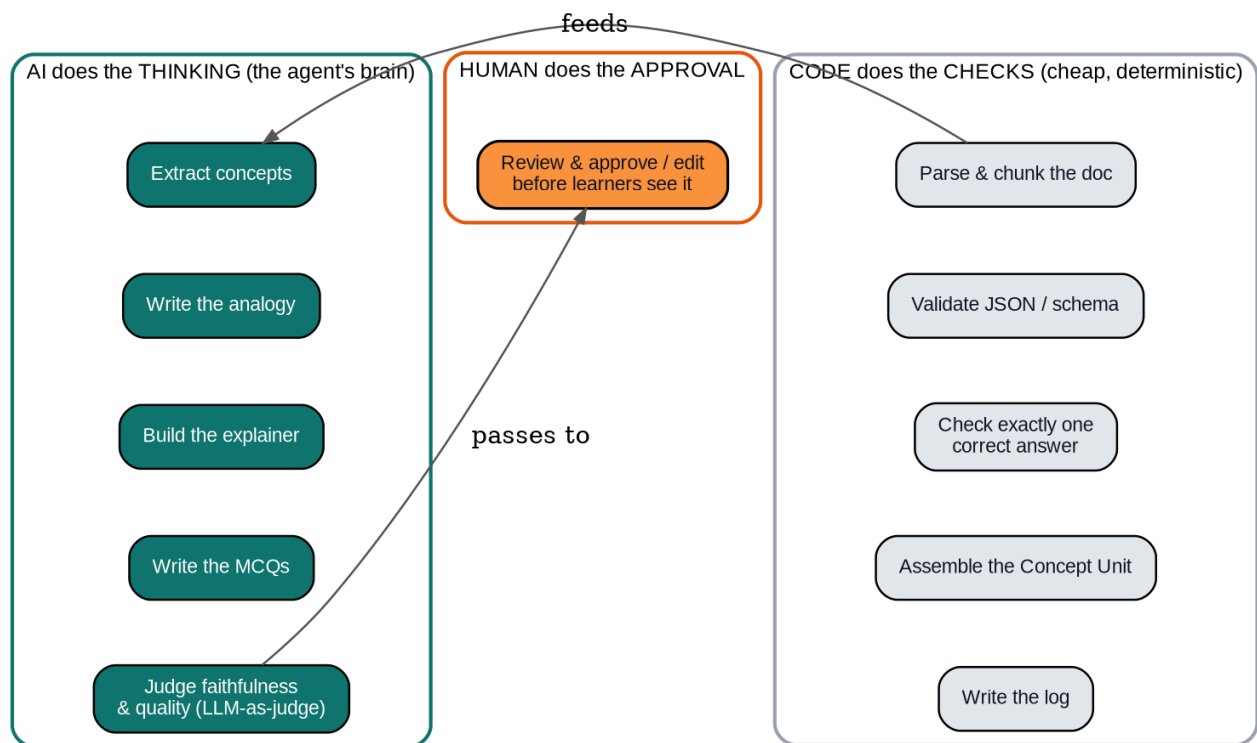
Stages in order:

1. **Input** — a static Markdown doc.
2. **Parse & chunk (code)** — split so each concept maps to a source span.
3. **Skill 1: concept-extraction (AI)** — produce the concept list.

4. **Eval gate 1** — coverage vs the gold list; fail → loop back, pass → continue.
5. **Per-concept loop:** retrieve span (*code/RAG*) → **Skill 2 analogy / Skill 3 explainer / Skill 4 MCQs** (*AI*) → assemble + validate (*code*) → **Skill 5 eval-audit** (*code graders + LLM-judge*).
6. **Eval gate 2** — pass rubric? fail → auto-retry (within limit) → regenerate.
7. **Human review gate** (*human*) — approve or edit. Nothing ships without this.
8. **Output** — render the interactive concept units on the platform.
9. **Memory** — style/conventions injected into generation; reviewer corrections fed back.
10. **Logging** — every run recorded (trace, eval scores, edits).
11. **Self-evolving loop** — each failure becomes a new eval case + prompt/grader fix, tightening the gates, grader, and memory over time.



5. AI vs code vs human



- **AI (thinking):** extract concepts, write analogy, build explainer, write MCQs, judge faithfulness/quality.
- **Code (cheap checks):** parse/chunk, validate schema, check one-correct-answer, assemble, log.
- **Human (approval):** review/approve/edit before learners see anything.

This split (code for rules, LLM-judge for meaning, human for taste) is the Session 2 graders design — it's *more* professional, not less AI.

6. The 5 skills

Skill	Input	Output	Done-when
concept-extraction	Raw MD	Concept list <code>{id, title, summary, source_span}</code>	Every key concept captured + linked to source

analogy-gener ation	Concept + span	<code>{concept_id, analogy, grounding_c heck}</code>	Faithful, clear, passes rubric
explainer-build er	Concept + summary	Renderable widget (SVG/HTML/R eact)	Learner can <i>see</i> the concept
mcq-generatio n	Concept + span	MCQs <code>{question, options, correct, explanation , difficulty}</code>	One correct, plausible distractors
eval-audit	All artifacts + eval set	Pass/fail report	Reviewer sees only what needs attention

Each skill = a SKILL.md (the instructions) + optionally a `scripts/` folder for its deterministic code.

7. Build steps

1. **Lock scope** — pick 1 doc; write the done-when paragraph.
2. **Design the output schema** — the exact JSON shape of one Concept Unit.
(*Everything depends on this.*)
3. **Set up access** — LLM API key working; pick generator + grader models; one test call.
4. **Parse & chunk** — script that loads the doc and maps concepts to spans.
5. **Build the eval set** — 5–10 real-failure cases + rubrics (*intern leads*).
6. **Skill 1 (extraction)** — run it; its output is the gold-list check.
7. **Skills 2 & 4 (analogy, MCQ)** — first generation pass.
8. **Skill 5 (eval-audit)** — code graders first, then LLM-judge.
9. **Skill 3 (explainer)** — lightweight widget (no video in V1).
10. **Orchestrator** — wire all steps + the human gate (plain Python).

11. **Render + log** — units on platform; logging sheet live.
 12. **Run a full eval set** — error analysis, fix top failures, retest.
-

8. Who does what?

- **Khushi** : schema, skill prompts, orchestrator, code + LLM-judge graders, retrieval, integration, rollout calls, the eval set (real failures + rubric pass/fail with your guidance)
 - **Pallavi**: gold concept list, human review of outputs, the front-end Concept Unit + MCQ widget, QA + failure logging.
-

9. The non-negotiables

1. **Human review gate** before publish (autonomy = Assist/Recommend, never auto-publish in V1).
2. **Faithfulness guardrail** — generated content must trace to the source; if a needed fact isn't there, flag instead of inventing.
3. **Eval set from real failures** — 5–10 cases, re-run before every change.
4. **Logging** on every run — it feeds the self-evolving loop.